Department of Computer Science
KISPECIxxx - Master Thesis

Thesis Title

Prepared by: Your name (xxxx@itu.dk)
Supervised by: Supervisor's name
Date: June 8, 2026

Abstract

This thesis proposes the design, implementation and evaluation of a survivable inspection UAV tailored for GPS-denied industrial environments. The project will develop a novel drone platform combining a lightweight, impact-tolerant protective structure that doubles as a contact sensor, an onboard perception stack for valve and inventory inspection, and a hybrid localization approach that fuses lidar/optical flow/IMU with UWB relays to enable robust close-proximity flight. Software work includes porting and integrating a PX4/ROS2 flight stack with Webots simulation to allow reproducible experiments, plus computer-vision pipelines for detecting valves, liquid levels and shelf items. The drone will be evaluated in simulation and physical tests on metrics including localization accuracy, inspection success rate, survivability (impact/drop resilience), and autonomy in docking/charging scenarios. The outcome aims to demonstrate that combining mechanical resilience with sensing-aware frame design and hybrid localization improves inspection reliability in confined, industrial settings — a practical step toward safer, more autonomous UAV inspection systems. The project aligns with the REAL LAB inspection topics and fulfils the MSc learning objectives by combining theory, tooling and empirical validation.

Will be now completelly different, will be industrial settings, cluttered environments, survivability

lets see

None of this

Like this is the only thing that should work like that lol

mostly ok, except for the localization -> lets see how the localization by hitting things actually work

Contents

1 Introduction

here should a be whole lot of text addressing the the thing in question bellow, mention the indusry case itself, mention the trouble of SLAM and obstacle detection and even the impossibility of perfect detection and avoidance.
MISSING WHOLE LOT

1.1 Research Question

Here should be clearly outlined what I am asking, what can be done, and how through which methods will I try to achieve it

Based on these challanges the goal of this thesis is to develop a drone for industrial settings which is survivable and is able to recover from clashes with its environment without breaking or finishing its mission early, autonomous enough to be able to navigate through signal denied environments in industrial settings and powerful enough to be able to carry the equipment helping it to both autonomously navigate and survive clashes with enough lift overhead capacity to allow for additional development, sensor mounting etc while being small enough to fit through manholes which it might need to navigate through for variuos inspections.

2 Related work

2.1 Collision-Resilient Drone Designs for Confined spaces

Here mention the congifly [16], spinfly[18] as something I mainly continued and learned from , Flyability[9] cluttered environmnet inspiration to how to build the cage [19] talk about gomboc and what not FOR NEXT TIME -> start easy on the first 2 things, perhaps talk about th edevelopment of the drone first in the Methodology section

This paper is a spiritual continuation of Spinfly [18] by A. Lerche. Lerche had a a similiar problem statement and he tried to develop a drone for autonomous inspection, in particular odometry. From his work and personal conversations with him a number of lessons to avoid in this work has been identified. Lerche used iNAV flight control framework as backbone of his drone development. While iNAV allows for autonomous flight it si not particularly well suited for fast development. Lerche had spent a considerable

time and resources on hardware connections, particularly soldering of Flight Controller and other devices together and various troubleshooting of hardware and software in iNAV. Constraint of navigating cluttered spaces safely brought me to work on Robust Aerial Swarms by [22] where instead of active obstacles detection and avoidance they instead crashed drones and have them recover from the crash, albeit with drone swarms each weighing 25grams. This takes a radically different approach to solving 'how to get from A to B' as Simultaneous localization and visualization (SLAM) is at a forefront of robotics and in many ways its holygrail, Mulgaonkar has instead chosen to map environment by clashing with it, and instead vicariously map it by localization of the drones themselves. And instead of relaying on detecting obstacle first and never clashing with it, have resilient enough drone which can instead recover from the clash and continue its mission. As SLAM is being developed it is not a ready technology for relatively small drones. Even best automakers in the world have still not brought this technology to navigate reliably any environment at the moment with state of the art LIDARs and other expensive and somewhat heavy equipment. While unmapped and changing environment of potential drone use in cluttered environment can be many fold more difficult to navigate and thus require quite sizable, heavy and expensive equipment if the drone was to go down the SLAM road for autonomy. While small drones in the aerial swarms could navigate their environment with clashing, using a a simple cage, the drone proposed in this work should be able of partially autonomous flying when being denied signal and even have ability to inspect its environment with various sensors it might use for it. This warrants some functional payload overhead which due to lift physics can be overcome only so much by more powerful motors, and at certain point the drone must grow in size. For this project a 4inch propeller drone design was chosen based on criteria mentioned later. At this size the penalty of hitting something increases substantially *DATA???? citetions? soemthing to proove this claim???* as the ratio of inertia and relative structural strenght of the drone and its propellers decreases, and on the other hand with the increased size of the drone everything in its environment is relatively smaller. Meaning that while Mulgaonkar could rely on simple 2 tube cage spanning their drones to avoid most items in their environment, the protection of a 4inch drone (4inch propellers -> at minimum 20cm in X and Y dimmensions) must be such that this drone is protected from corners of items such as shelves or continuous pipping and that such objects do not directly interfere with the drone, and with its propellers in particu-

lar. Here a research 19, Development of Collision Resilient Drone for Flying in Cluttered Environment helps to tackle this issue. He proposes either a gimbal like sphere which freely spins upon contact with its environment, similiar as a commercial product by Flaybility [9] *Sourcing missing, fucked up ZOTERO* and a Gombok [3] *eve fucking wikipedia reference will do here.* Gombok shape, or in this case a turtle-like shape has an interesting property of being potentially stable only in one orientation, when a turtle is turned upside down it can just by its own movement come back to its feet thanks to its center of gravity being suddenly high up in such case, making it more unstable. Mahmout has laveraged these properties in their design as well and concluded his paper with it being the superior cage design of the two. A specific use case given by supervisor of this paper comes to mind: inspection of stainless steel tanks. Today this is executed by a human who needs to descent into the tank through a menhole on the top. Hypothetically a turle shaped drone cage, which would after inspecting the tank ascend to fly out through the menhole, could more easily get through the narrow opening even if it would not navigate perfectly out of the hole, as if only the center of the drone would be within the manhole limits the drone could somewhat stably ascend without it being rocked too hard, as it would leverage its shape and its low center of gravity. Due to this hypothesis, the resulsts Mahmut had with their cage design, relative simplicity opposed to gimbal sphere and even possibility of using it similarly as Lerche did for odometry, the spinning turtle shape was eventually chosen for the drone design.

Apart from physical survivability of larger drones in cluttered environments a topic of software control for autonomous flight had to be addressed. While this is a frontier technological development topic, 2 autonomous flight platforms standout with their rate of adoption and technological maturity, ArduPilot and PX4. Both are widely adopted and have a lively base of developers who add to their codebases. Apart from more free licensing of PX4 (which even allows for commercial extension based on the PX4 for proprietory purposes, while Ardupilot requires all development to be added to their codebase CITATION MISSING AGAIN) PX4 differs from Ardupilot having more modularized architecture and having more well developed hardware standards. This gave PX4 the edge in this work as it allowed for more straightforward attitude to the development and using standardized parts presumably with no or little troubleshooting.

3 Methodology

The methodology for RATFLY project stands on development of drone to use for the various tests and implementations together with controlled environment to test the drone in. As to the development, it consists of hardware choices, software choices and the constraints sprouting from these two which dictate much of the development.

3.1 Design and Development

3.1.1 Flight Type

A quadcopter design was a very clear choice as these are widely used and allow for very nimble and precise movement as opposed to fixed-wing designs which need to be in forward motion to generate upthrust for their flight. The Quadcopter design due to its symmetric nature make it very easy to predictably control it.

On figure Figure 1 you can see standard quadcopter setup. If front motors (m_1, m_2) spin comparatively more the drone pitches up, if side propelers (m_1, m_4) spin comparatively faster, then the drone rolls towards its left side, and thanks to the setup of alternating spin direction if propellers diagonally from each other fx m_1 and m_3 will spin faster, while m_2 and m_4 will spin slower the drone will turn on around in the direction of the propolers which are faster, in this case turn towards its left. This is given by Newton's first law as the inertia of propellers spinning faster will induce the rotation in the propellers' direction. Opposingly, pitch and roll is achieved by spinning pairs of propellers on one side faster than on the other (left vs right, front vs back). This makes the control of quadcopter somewhat simple as it can be summarized in a mathematical model.

3.1.2 Functional constraints and considerations

As this work is in certain sense a continuation of Lars [18] Spinfly paper, lessons learned from his project were well considered. One of the largest limitations mentioned in the work and also by the author himself in person was insufficient weight headroom of the drone which had trouble taking off and maintaining the flight when fitted with the cage. Another one was insufficient battery life which did not allow for prolonged flight and made

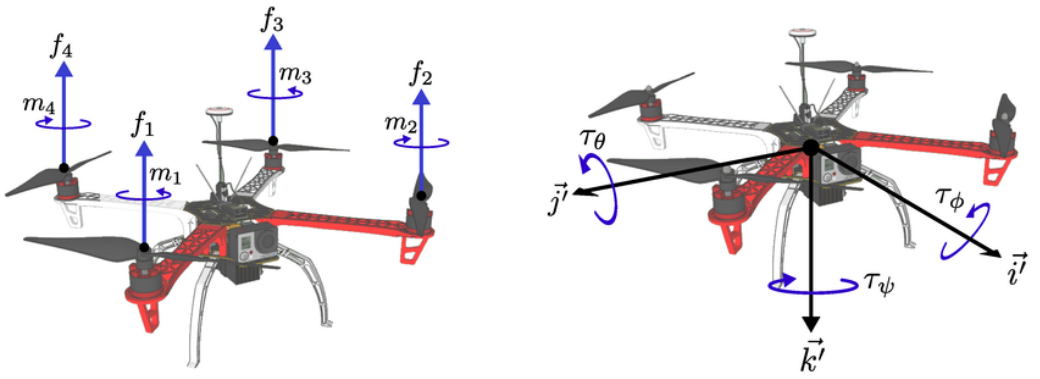

Figure 1: typical quadrocopter setup 'props-in', yaw ψ (turning around), pitch θ (nose up or down) and roll ϕ (tilting to sides) From [17]

subsequent tests difficult to run. Both of these problems are solved in tandem as battery capacity and its weight are directly proportional and can have diminishing returns if the battery has high capacity but get drain faster as the motors have to output more power. Lerche also reflected on lack of robust autonomous controller on board and troubles with telemetry realying the communication. [I want to make this more structured, maybe better extract the infomarion, maybe summup that was I needed to do was

1. Carry bigger load
2. Fly longer/more efficiently
3. Simpler and more robust flight control [FC]

Size of 6C FC is substential enough to warrant a 4inch drone design, while typically drones of this size, 4inches is also the upper maximum size to be flown indoors [2]. The ratio of trust-to-weight for common drones is between 4:1 to 3:1, meaning maximum thrust of the the motors combined should be 3-4times greater than the weight of the drone. Due to needed weight headroom, which by estimation and after discussion with Spinfly author, I set to be 800-1000g. I chose Emax ECOII 2004 1600KV motors. These are typically recommended for 4.5inch to 5inch ultalight drone builds, but similiar motor by a different company has been tested with 4inch propellers GF4023 propellers and proven to 860g of thrust per motor, combined 3440g per drone giving me 1:3.44-1:4.30 ratio depending on the total weight. This also means

the whole drone can stay hovering with approximately 50% of thrust which is recommended due to the motors being twice as efficient than at 100% trust. Designation 2004 means it has 20mm diameter and 4mm in height, this is significant diameter in the class which makes it slower to respond and adjust RPMs, on the other hand it allows for higher trust at 50% throttle. 1600KV metric is how many RPMs are produced when applying 1V. 1600KV is at the low end of KV spectrum and was selected in accordance with using a 6 cell battery which provides 24V providing ca 38400RPM (1600 * 24) which falls well within recommended speeds for 4inch propellers. 6S battery is also an important choice from the efficiency perspective as running higher voltages draws less current and the whole settup keep a lower temperature as opposed to 4S setups. Additionally there is voltage sag phenomenmon (LLM here-> explain it for me pls) that is less of a factor with higher voltages settups To address this holistically with a focus on the my research question I choose PX4 standard for my autonomous flying solution. PX4 is a an open source standard with modular architecture uORB (Micro Object Request Broker) which is suited for plug-and-play development as it's high modularity allows for faster research, as the physical Flight Controller (FC) manufactured under this standard Pixhawk 6C, as opposed to most FCs, so the accompanying electronics can be quickly swapped, tried and iterated on. This Pixhawk 6C FC has been chosen as a solid FC solution with IMU and compass sensors already integrated, simplifying the drone development and allowing me to concentrate on the research question itself. [I feel like this should be]

Iterative Learning from Predecessor Work

The design began with critical constraints identified in the Spinfly platform [18]. Two primary limitations directly shaped this work:

1. **Insufficient weight headroom** – The original platform struggled with takeoff and stable flight when equipped with the protection cage, leaving no margin for component upgrades or mission adaptations.

2. **Insufficient battery capacity** – Limited flight time created operational friction; test iterations became difficult to schedule, reducing research throughput.

These problems are **not independent**—they form a coupled system. Battery capacity and its mass are directly proportional, yet simply increasing capacity can create diminishing returns. A heavier battery forces motors to

output more power, which accelerates discharge rate, potentially wasting the added capacity. The design needed to solve both simultaneously through systematic trade-off analysis.

Lerche also highlighted secondary constraints: lack of robust autonomous flight control on the flight controller itself, and telemetry transmission reliability. These informed the autopilot platform selection.

Design Objectives: Based on these constraints, the design targets were clarified into three interconnected goals:

1. **Carry larger payloads** – The base payload must be much larger, to allow for cage development and to leave weight headroom for any additional development
2. **Achieve longer flight duration** – Extend mission time while maintaining efficient power consumption.
3. **Simplify autonomous control** – Use a proven, modular flight controller stack to reduce software complexity and improve reliability.

Platform Selection: PX4 and Pixhawk 6C To address these objectives holistically, PX4 was selected as the autonomous platform. PX4 is an open-source standard built on modular architecture using **uORB (microObject Request Broker)**—a publish-subscribe middleware layer suited for plug-and-play development.[21] This modularity enables rapid research iteration: novel autonomous behaviors can be implemented as independent modules without modifying the core flight stack.

The **Pixhawk 6C flight controller** was chosen as the hardware instantiation of the PX4 standard. Critically, Pixhawk is not a monolithic design but represents standardized connector specifications, sensor integration points, and reference schematics managed by the Dronecode Foundation. This standardization means:

- **Hardware swappability**: Sensors and peripheral electronics can be upgraded or replaced with components from different manufacturers (Holybro, ARK Electronics, CUAV) without redesigning integration.
- **Ecosystem maturity**: Over 1 million Pixhawk-based devices are deployed across commercial, research, and hobbyist applications.[27]

- **Sensor integration**: The Pixhawk 6C includes integrated IMU and compass. Specifically, it features redundant inertial measurement units with onboard ICM-42688-P and BMI088 accelerometers/gyroscopes, plus an IST8310 magnetometer, with temperature-controlled operation via onboard heating resistors.[5–7] This eliminates the need for custom sensor fusion code—allowing focus on the research question rather than flight control tuning.

The 6C's form factor (84.8 × 44 × 12.4 mm) justifies the 4-inch drone design choice.[7] Four-inch platforms represent the upper practical limit for indoor flight while remaining large enough to carry meaningful payloads.

Thrust-to-Weight Ratio: Balancing Performance and Efficiency Definition and Target Range

Thrust-to-weight (TWR) ratio expresses the total available thrust divided by drone mass. It directly determines hover efficiency and responsiveness. Industry guidelines establish a spectrum across applications.[23]

Application	TWR	Hover Throttle	Notes
Aerial photography	2:1 to 3:1	40–50%	Stable, efficient
Research/cargo	3:1 to 4:1	30–50%	Responsive, good efficiency
Racing/freestyle	5:1+	15–25%	Aggressive maneuvers

Why TWR Matters: Efficiency and Control Authority

Motor efficiency does not follow a simple linear relationship with throttle percentage. Instead, efficiency curves show a **peak efficiency at mid-range throttle** (typically 40–70%), with degradation at both very low and very high throttle levels.[8] This is critical for autonomous drones.

If a drone hovers at 100% throttle, it has zero authority to climb, stabilize, or reject wind gusts—any disturbance forces maximum output, leaving no control margin and risking loss of stability during maneuvers.[14] Conversely, hovering at 40–50% throttle reserves substantial available thrust for autonomous obstacle avoidance, gust rejection, and rapid maneuvers required during precision autonomous flight.[26]

Motor Selection: EMAX ECOII 2004 1600KV Why This Motor?

The EMAX ECOII 2004 series is specified for 4–5 inch ultralight builds.[1] The 2004 designation specifies stator dimensions: 20mm diameter and 4mm

height.[1] This dimensional choice has performance implications explained below.

Pilot testing of a similar 2004-series motor (T-Motor F2004 1700KV, comparable in stator dimensions) paired with 4-inch GF4023 propellers and a 6S battery demonstrated approximately 860 grams of static thrust per motor.[15, 23] With four motors, this yields combined thrust capacity of approximately 3,440 grams—establishing the system's TWR.

TWR Calculation (estimated final weight 800–1,000g):

- Total available thrust: 3,440g
- Target weight: 900g (midpoint)
- TWR = 3,440 / 900 = **3.82:1**

This places your design in the research/autonomous platform zone. A TWR of 3.8:1 means the drone can sustain controlled hovering well within the efficient mid-throttle operating range.

Understanding Motor Nomenclature

The **2004 designation** specifies 20mm stator diameter and 4mm height. Larger stator diameters generate more torque but have slower transient response (RPM changes take longer to complete) compared to smaller-diameter motors.[20] This trade-off is deliberate: at mid-throttle operation, the 2004's larger stator sustains higher efficiency than smaller, higher-KV alternatives that must spin faster to produce equivalent thrust. For autonomous flight requiring smooth, predictable thrust response, this characteristic is desirable.

The **1600 KV rating** indicates 1600 RPM per applied volt. This low-KV designation was selected specifically for your 6S battery voltage.

Battery Selection: 6S LiPo Why 6S and Not 4S?

Battery voltage selection cascades through the entire drivetrain. The **6S (six-cell) LiPo pack** produces nominal voltage of 22.2V (6 cells × 3.7V per cell).[11] This voltage selection determines optimal motor KV and operating efficiency.

1600 KV with 6S Battery:

- Theoretical no-load RPM: 1600 KV × 22.2V = **35,520 RPM**[8]
- Practical RPM under propeller load: ~24,000–28,000 RPM (typically 70–80% of theoretical no-load speed due to air resistance and propeller drag)[23]

- This range is optimal for 4-inch propellers, avoiding excessive tip speeds that reduce aerodynamic efficiency

Higher voltage = less current for equivalent thrust. When motors operate on 4S (˜14.8V nominal), they require significantly higher current draw to achieve the same thrust.[24] Higher current increases internal losses in battery, ESCs, and motor windings—manifesting as excess heat and reduced flight time. 6S draws proportionally less current for equivalent thrust, keeping system temperature lower. This is measurable: one study comparing 4S vs 6S motor systems on the same mechanical load found that 6S exhibits "greater low-end control" with lower current heat generation under identical thrust demands.[25]

Voltage Sag: The Critical Factor for Autonomous Control What is Voltage Sag and Why It Matters

Voltage sag refers to the **temporary voltage drop that occurs during high-current discharge events**. This drop is caused by battery internal resistance (IR). Ohm's Law describes this relationship: **Voltage Drop = Current × Internal Resistance**.[10]

Every LiPo battery contains internal resistance—even high-quality cells. When you demand high current from the battery (e.g., during rapid acceleration), that current flowing through internal resistance creates a voltage drop.[24] For example:

- A 4S battery (14.8V nominal) with typical internal resistance might sag from 14.8V down to **10.2V** during peak acceleration
- A 6S battery (22.2V nominal) with equivalent quality, under the same peak current draw, experiences proportionally less sag (from 22.2V down to ˜16.5V)[4]

The absolute sag magnitude (difference in volts) may be similar between 4S and 6S. **But the percentage impact is vastly different:**

- 4S sag example: 14.8V → 10.2V is a **31% voltage loss**
- 6S sag example: 22.2V → 16.5V is only a **26% voltage loss**

Why This Matters for Autonomous Flight Controllers

Your Pixhawk 6C relies on stable voltage to maintain consistent motor response during rapid maneuvering. The flight controller sends throttle commands expecting motors to accelerate at a predictable rate. **Excessive voltage sag introduces latency between commanded acceleration and actual motor output.** Here's the causal chain:

1. Flight controller commands 70% throttle for obstacle avoidance maneuver
2. Motor voltage sags unexpectedly from 22V to 16V due to high current draw
3. Motor RPM drops below what the controller expected
4. Actual thrust output is lower than the controller's state estimator predicted
5. The control loop detects the mismatch and must recalculate—introducing lag

During high-speed autonomous maneuvers, this lag degrades control stability and can risk uncommanded behavior. Higher baseline voltage (6S) provides a larger voltage margin, reducing the percentage impact of sag on motor performance.[24]

6S Enables Efficient Hover at Higher Throttle Margin

Since 6S provides higher baseline voltage, your motor's hover throttle point is positioned differently on the efficiency curve compared to 4S. Hover throttle percentage depends on specific motor and load characteristics—**there is no direct proportionality between voltage and hover throttle**. However, empirical testing shows: **"6S is a clear winner in efficiency up to around 45 throttle [percentage]."**[25]

Your 1600KV motor on 6S should hover in the **45–50% throttle range** based on your estimated drone weight and motor thrust.[20] This operating point—hovering near the motor efficiency peak—means you extract maximum flight time from your battery while maintaining responsive control authority (50% throttle reserves 50% available thrust for maneuvers and environmental disturbances).

System Integration: How Components Work Together This configuration is tightly coupled—changing any single parameter cascades inefficiencies throughout the system:

Parameter	Your Design	Rationale
Frame size	4-inch wheelbase	Pixhawk 6C form factor (84.8 × 44 × 12.4 mm) + indoor flight compliance
Motor KV	1600 KV	Produces 24,000–28,000 RPM under load on 6S; optimal for 4-inch propellers[23]
Battery voltage	6S (22.2V nominal)	Reduces voltage sag percentage impact; improves efficiency at cruise throttle[25]
Hover throttle	~45–50%	Operates at motor efficiency peak; reserves 50% available thrust[20, 25]
Estimated TWR	3.82:1	Responsive yet efficient for research autonomous missions

If you switched to 4S battery, for example, you would need lower-KV motors (e.g., 1100KV) to maintain similar RPM. But lower-KV motors produce less torque, potentially requiring larger propellers or higher current draw—both of which degrade efficiency. Every selection interconnects. Your component choices represent a validated system point.

4 Physical design and considerations

The

5 Autonomous Drone Companion Computer Setup Workflow

5.1 Overview

Setup a Raspberry Pi 5 running Ubuntu 24.04 with ROS2 Humble in Docker containers for autonomous drone development with PX4 flight controller integration. The containerized approach solves Ubuntu 22.04/Pi 5 kernel incompatibility while providing ROS2 Humble support.

5.2 Phase 1: OS and Network Setup

Flash Ubuntu 24.04 Desktop to microSD using Raspberry Pi Imager with the following configuration:

- Hostname: `pi5drone` (appears as `dorten-pi5drone` in system)
- Username and password configured
- SSH enabled via Services tab (critical step)
- Wi-Fi configured for local network
- Locale: Europe/Copenhagen, Country: DK

Note: Raspberry Pi 5 uses micro-HDMI ports for display output, not USB-C. USB-C is power-only (27W PSU recommended).

Boot the Pi and verify connectivity. SSH server is not auto-started despite Imager setting—manual installation and enablement is required:

```
sudo apt install openssh-server
sudo systemctl enable --now ssh
```

5.3 Phase 2: mDNS Hostname Resolution

Problem: Connecting via `ssh dorten@dorten-pi5drone.local` failed with name resolution errors.

Solution: Install Avahi daemon on both Pi and workstation for multicast DNS (mDNS) support.

On Raspberry Pi:

```
sudo apt install avahi-daemon
sudo systemctl enable --now avahi-daemon
```

On workstation (Ubuntu):

```
sudo apt install avahi-daemon libnss-mdns
sudo systemctl enable --now avahi-daemon
```

Verify hostname resolution:

```
ping dorten-pi5drone.local
```

Optional enhancement—Passwordless SSH: Execute `ssh-copy-id dorten@dorten-pi5drone.local` from workstation (one-time operation eliminates password prompts for subsequent connections).

5.4 Phase 3: Docker Installation

Install Docker Engine following official documentation [12]:

```
sudo apt install docker.io docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Known issue: Ubuntu 24.04 ships with `containerd` package that conflicts with Docker's `containerd.io`. Remove before installation:

```
sudo apt remove containerd
```

Configure rootless Docker to allow non-root user container execution [13]:

```
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
```

Verify installation:

```
docker run hello-world
```

5.5 Phase 4: Python Virtual Environment

Ubuntu 24.04 implements PEP 668 externally-managed-environment policy, blocking system-wide pip installations. Create isolated Python environment:

```
sudo apt install python3.12-venv
python3 -m venv ~/ros2_env
source ~/ros2_env/bin/activate
pip3 install vcstool
```

Note: Virtual environment activation required for each session, or can be configured in shell startup files.

5.6 Phase 5: ROS2 Humble in Docker Containers

Rationale: Ubuntu 22.04 is the official platform for ROS2 Humble binary packages. Direct installation on Raspberry Pi 5 running Ubuntu 24.04 is not officially supported due to kernel incompatibility with RP3A0-M1 hardware. Containerization provides Ubuntu 22.04 environment running inside the 24.04 host, enabling native ROS2 Humble support without source compilation.

Create workspace structure:

```
mkdir -p ~/ws/.devcontainer
cd ~/ws
```

Create `Dockerfile` providing base image with Ubuntu 22.04 + ROS2 Humble (using official `ros:humble` image from Open Source Robotics Foundation), non-root user account with UID matching host system, passwordless sudo configuration, and development tool installation.

Create `.devcontainer/devcontainer.json` defining Visual Studio Code container configuration. Key settings:

- `-net=host`, `-pid=host`, `-ipc=host`: Required for ROS2 DDS discovery
- `ms-iot.vscode-ros`: ROS2 syntax highlighting and tooling
- `privileged: true`: Enables GPIO and UART access for PX4 flight controller communication
- X11 forwarding mounts: Support for graphical applications (RViz2, rqt)

- `postCreateCommand`: Executes rosdep dependency resolution, workspace permission fixes, and ROS2 environment setup

5.7 Phase 6: VSCode Remote Development

On development workstation:

1. Install "Remote - SSH" and "Dev Containers" extensions in Visual Studio Code
2. Command Palette (Ctrl+Shift+P) → "Remote-SSH: Connect to Host" → `dorten@dorten-pi5drone.local`
3. VSCode opens remote window connected to Pi
4. Open folder `/home/ws`
5. VSCode detects `.devcontainer/devcontainer.json`, prompts "Reopen in Container"
6. VSCode builds Docker image (first execution: approximately 30–50 minutes on Raspberry Pi 5)
7. Container starts and initializes; development environment ready

Verification of containerized environment:

```
lsb_release -a          # Expected: Ubuntu 22.04 LTS
ros2 --version          # Expected: ROS 2 Humble
echo $ROS_DISTRO        # Expected: humble
```

5.8 System Architecture

The complete development architecture is illustrated in Figure 2.

5.9 Key References

- Docker installation: `https://docs.docker.com/engine/install/ubuntu/`
- Rootless Docker configuration: `https://docs.docker.com/engine/install/linux-postinstall/`

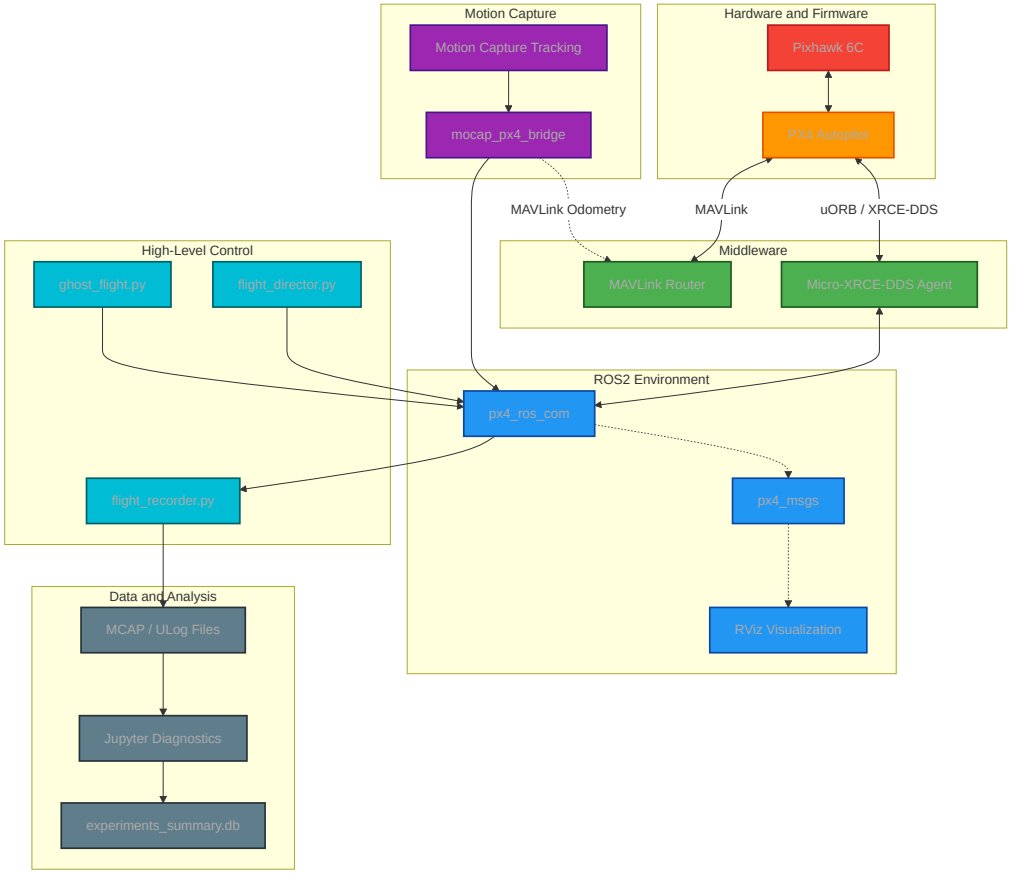

Figure 2: Drone software stack architecture with PX4 flight controller, ROS2 middleware, MOCAP integration, and data analysis pipeline

- ROS2 Docker VSCode guide: `https://docs.ros.org/en/humble/How-To-Guides/Setup-ROS-2-with-VSCode-and-Docker-Container.html`
- VSCode Remote SSH: `https://code.visualstudio.com/docs/remote/ssh`
- PX4 companion computer: `https://docs.px4.io/main/en/companion_computer/pixhawk_rpi`
- uXRCE-DDS middleware: `https://docs.px4.io/main/en/middleware/uxrce_dds`

6 Next Steps: PX4 Integration

6.1 Hardware Setup: Pixhawk to Pi GPIO Connection

Connect Pixhawk TELEM2 port to Raspberry Pi GPIO pins for UART serial communication (921600 baud):

Pixhawk TELEM2	Pi GPIO Pin	Function
TX (pin 2)	GPIO 15 / RXD (pin 10)	Receive data
RX (pin 3)	GPIO 14 / TXD (pin 8)	Send data
GND (pin 6)	Ground (pin 6)	Common ground

Table 1: Pixhawk TELEM2 to Raspberry Pi GPIO wiring for uXRCE-DDS communication

Verify UART is enabled on Raspberry Pi and that device node `/dev/ttyAMA0` is accessible to the container user.

6.2 Install uXRCE-DDS Agent

Inside VSCode terminal (already within container):

```
sudo apt install ros-humble-micro-xrce-dds-agent
```

The uXRCE-DDS Agent acts as a bridge between the PX4 flight controller (running uXRCE-DDS client) and the ROS2 middleware, translating between uORB messages and ROS2 topics.

6.3 Launch uXRCE-DDS Agent

Start serial connection to Pixhawk:

```
MicroXRCEAgent serial --dev /dev/ttyAMA0 -b 921600
```

Expected output: Agent status indicating successful session creation once Pixhawk boots and establishes connection.

6.4 Verify PX4-ROS2 Communication

In new VSCode terminal (container), list active ROS2 topics:

```
ros2 topic list
ros2 topic echo /fmu/out/vehicle_odometry
```

Expected: Topics from PX4 flight controller such as `/fmu/out/vehicle_odometry`, `/fmu/out/vehicle_status`, `/fmu/out/vehicle_attitude`, etc.

6.5 Configure PX4 Parameters

Using QGroundControl on development workstation, configure PX4 firmware parameters to enable uXRCE-DDS middleware:

- `UXRCE_DDS_CFG`: Set to TELEM2 (serial port configuration)
- `SYS_AUTOSTART`: Select appropriate vehicle type (quadcopter, fixed-wing, etc.)
- Verify serial connection parameters match uXRCE-DDS Agent configuration (921600 baud)

6.6 Develop ROS2 Autonomous Control Nodes

Implement mission planning and autonomous flight logic in ROS2:

- Subscribe to PX4 telemetry topics: `/fmu/out/vehicle_odometry`, `/fmu/out/vehicle_statu`
- Publish control commands to PX4: `/fmu/in/offboard_control_mode`, `/fmu/in/trajectory_setpoint`
- Implement waypoint navigation, obstacle avoidance, and mission state machines in Python or C++
- Integrate sensor inputs (GPS, IMU, cameras) from PX4 uORB message stream via ROS2 topics

6.7 Testing Methodology

1. **Simulation First:** Use PX4 Software-in-the-Loop (SITL) simulator to test autonomous algorithms without real hardware risk
2. **Monitor Telemetry:** Subscribe to PX4 topics in ROS2 to verify correct message flow and telemetry accuracy
3. **Hardware Validation:** Deploy ROS2 nodes to Raspberry Pi container once simulation testing successful
4. **Flight Testing:** Monitor autonomous flights via MAVProxy or QGroundControl; collect flight logs for analysis

6.8 SSH Authentication Setup for Containerized Development

Purpose: Enable Git operations from within a Docker container using host SSH credentials.

Challenge: Docker containers are isolated environments. By default, a container cannot access the host system's SSH keys needed for GitHub authentication.

Solution implemented in three steps:

6.8.1 SSH Key Generation (Host System)

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

Generated an ED25519 key pair on the Raspberry Pi 5 host system. The public key (`id_ed25519.pub`) was added to GitHub account settings under SSH keys, enabling passwordless authentication.

6.8.2 Container Configuration (devcontainer.json)

```
"mounts": [
  "source=${localEnv:HOME}/.ssh,target=/home/${localEnv:USER}/.ssh,
   type=bind,readonly"
]
```

Mounted the host's `.ssh` directory into the container as read-only. This allows the container to access SSH keys without duplicating or exposing them.

6.8.3 SSH Client Installation (Dockerfile)

```
RUN apt-get update && apt-get install -y openssh-client && \
    rm -rf /var/lib/apt/lists/*
```

Added OpenSSH client to the container image, enabling Git to use SSH protocol for remote operations.

Result: The development container can authenticate with GitHub using the host's SSH keys, enabling seamless `git push`, `git pull`, and `git clone` operations without credential prompts.

Why SSH over HTTPS: SSH keys provide secure, passwordless authentication and persist across sessions, eliminating the need to store tokens or enter credentials repeatedly during development.

6.9 Troubleshooting Reference

7 Experiments

7.1 Collision of Drone with Column

In this experiment, the primary objectives are to investigate the following:

- Whether a freely rotating cage has a measurable effect on the drone's ability to maintain or recover its desired flight path post-collision, compared to an affixed control cage.
- Whether the flight controller's internal Inertial Measurement Unit (IMU) can accurately predict the vector of impact, which could assist navigation by estimating the relative position of surrounding obstacles.

7.1.1 Physical Environment and Setup

The physical experiments were conducted at the REALLAB at the IT University of Copenhagen (ITU). The environment utilizes an OptiTrack Motion Capture (MOCAP) system consisting of six cameras, which were recalibrated

Issue	Likely Cause	Resolution
SSH connection fails	SSH daemon not running or Avahi not active	`sudo systemctl status ssh` and `sudo systemctl status avahi-daemon`
Docker build very slow	Normal on Pi 5 ARM64 architecture	First build 30–50 min; subsequent builds cached
ROS2 commands not found	Environment not sourced	Verify `postCreateCommand` in devcontainer.json
uXRCE-DDS won't connect	UART wiring, permissions, or baudrate mismatch	Check GPIO wiring, verify `/dev/ttyAMA0` exists, confirm 921600 baud
ROS2 topics empty	uXRCE-DDS client not running on Pixhawk	Verify PX4 parameters `UXRCE_DDS_CFG` set to TELEM2

Table 2: Common issues and resolution strategies

just few days prior to the testing sequence to ensure high-fidelity spatial ground truth.

For the collision obstacle, a vertically oriented cylindrical column was constructed using a cardboard roll. This roll was internally reinforced with an aluminum profile secured directly to both the floor and the ceiling of the laboratory. While the column oscillates slightly when subjected to manual force, no additional structural supports were added; this strict constraint was necessary to prevent any occlusion of the MOCAP cameras, preserving their function as the Single Source of Truth (SSoT) for the drone's spatial positioning.

7.1.2 Software Architecture and Control Pipeline

The experimental platform relies on a custom software pipeline bridging high-level path planning with low-level flight mechanics. The low-level flight controller operates on PX4 firmware built on a NuttX real-time operating system (RTOS), with internal state estimation processed exclusively through an Extended Kalman Filter (EKF2).

High-level position commanding and abstracted trajectory generation are managed via a ROS 2 abstraction layer running on an external offboard computer. This architecture requires the continuous streaming of setpoints to the flight controller. Critical topic communication and data serialization between the ROS 2 node graph and the internal PX4 controller are facilitated by a micro-XRCE-DDS agent acting as the requisite middleware bridge.

7.1.3 Mission Execution and Data Logging

Prior to the physical tests, a custom component named `flight_director.py` was developed to take control of the execution of the different experimental missions. This director actively enforces geofence safety rules to constrain the drone within the testing volume.

The mission control pipeline with flight director supervision, collision event recording, and telemetry flow is illustrated in Figure 2.

The flight director automates the data collection pipeline by initializing synchronized telemetry recordings, saving the flight logs directly into `.mcap` files. To isolate specific events, these files are strictly segmented by individual mission loops. Post-flight data extraction is handled by a custom Python database pipeline, which parses the `.mcap` files to compute, filter, and structure the physical variables required for the resulting figures and plots.

7.1.4 Flight Trajectory Protocol

The standard flight protocol requires the drone to take off and transition from an Exp. Start-point to an Exp. End-point. These coordinates are mapped as a deliberate straight-line trajectory that strictly intersects the static coordinates of the obstacle column. The primary measurable data includes the adherence to the desired flight path (rotating vs. fixed cage) and the high-frequency disturbance of the IMU unit in both configurations. This protocol was executed across a predefined distribution of incident angles.

7.1.5 Data Categorization and Analytical Variables

Impact events are systematically categorized into four primary bins based on the angle of incidence: 30°–40°, 40°–50°, 50°–60°, and 60°–90°. The custom Python pipeline extracts the following core metrics to evaluate mechanical performance:

- **Maximum Deceleration Profiles:** Evaluated at the onset of physical impact with the battery at nominal starting voltage. The fixed cage baseline establishes the instantaneous kinetic energy dissipation, determining if the rotating structure actively lowers peak deceleration forces transmitted to the core mass.
- **IMU Peak Acceleration (Z-Axis) vs. Commanded Motor Speed (RPM):** Isolates vertical force transfer relative to motor output. Data indicates the rotating cage registers a higher mean Z-axis acceleration ($\sim 9.5\,\mathrm{G}$) compared to the fixed cage control ($\sim 6.5\,\mathrm{G}$), despite operating at lower commanded motor speeds.
- **Acute Shock Phase Portrait:** Maps peak impact acceleration (X-axis) against the system's rotational energy (Y-axis). This evaluates the relationship between the kinetic energy absorbed by the cage's rotation and the residual shock transmitted to the central drone mass.
- **Post-Impact Y-Axis Vibration Spread:** A standard deviation analysis of lateral acceleration (A_y in units of G). The rotating cage demonstrates localized, tightly bound variances (0.3 G–0.6 G) compared to the high-amplitude jitter in the fixed cage (0.6 G–1.7 G), quantifying a distinct lateral stabilization effect from the independent rotating mass.
- **Recovery Area Path Deviation:** Quantifies the spatial deviation from the commanded flight path post-impact. At shallow incident angles (30°–40°), the rotating cage yields a temporarily larger mean recovery area. This represents a mechanical "rolling" interaction: the rotating structure maintains continuous physical tracking along the column's circumference, pressing into the obstacle to fulfill the straight-line command. This induces a directional trajectory overshoot once the geometry is cleared. In contrast, the fixed cage exhibits immediate elastic rebound, bouncing away and allowing a mathematically quicker, albeit higher-shock, return to the path vector.

7.1.6 Frame design

Hypothesis as opposed to previous works, confirmed by mahmoud

7.1.7 Cage design

mahmoud veci

7.2 Hardware

7.3 Software

8 Experiments

8.1 Collision of Drone with column

in this experiment I am tring to see a number of things: If rotating cage has an effect on the drone following in its desired fly path if the drone's flight controller's IMU is able to predict the vector of impact of the drone. Which could help with its nabigation by estimating where its surrounding obstacles are

The experiment has been conducted at REALLAB at ITU. For the experiment I used an Optitrack MOtion Capture system with 6 cameras which have been recalibrated less than 1 week prior to the testing. F

Prior to the experiments flight_director.py has been developed to take control of execution of different experiments called missions (Figure 3). The Flight director included a call to record these flights and automatically save these logs as .mcap files which were then dissected, analyzed and are the data for all the figures and plots bellow

8.1.1 Collision Experiment

The idea of this experiment was to test whether a rotating cage has a measurable impact on performance of the drone colliding with an obstacle. Due to this also affixed in place cage was used as control. For simplicity a cylinder column vertically oriented was chosen. For the obstacle I used a cardboard roll which has been affixed both from bottom and the top and had an aluminum profile going through which was secured both to the floor and also the ceiling of the room. the obstacle column would move / oscilate slightly when tested by hand, but no more structural supports were added in order not to

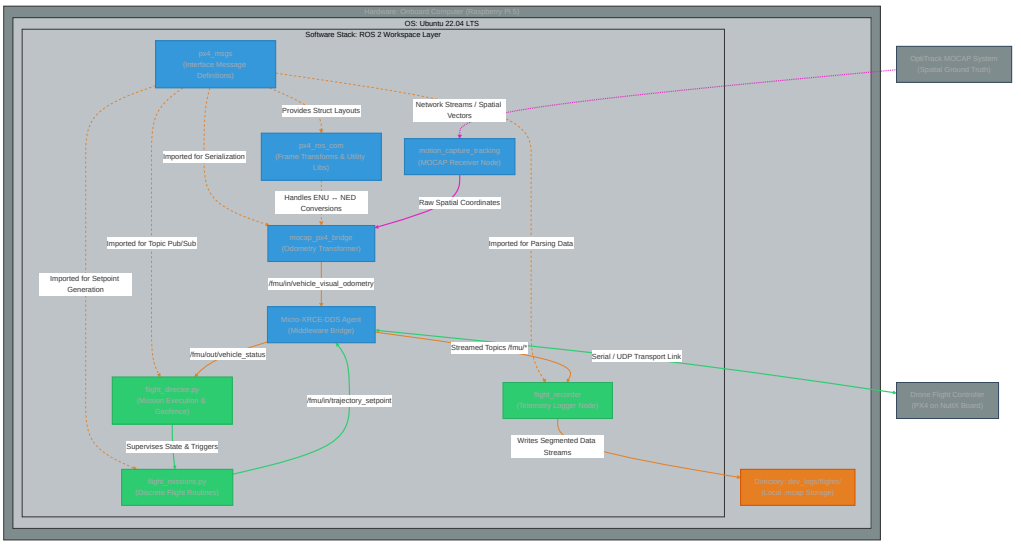

Figure 3: Experiment pipeline showing hardware components, ROS2 software stack, data flow from MOCAP through the agent to PX4, and telemetry logging to MCAP storage

obscure the MOCAP cameras which were the SSoT for the drone. (*Include pictures of the settup here, perhaps one with drone in it as well*) The Drone for this experiment needs to take off and then fly from Exp. Start-point to Exp. End-point which is delibrately chosen to be in a straight line where the drone if flown according to the desired waypoint must encounter the obstacle. As measurable data was chosen the adherence to the desired fly path of the drone with rotating cage vs with affixed cage and the disturbance of the IMU unit in both of these cases this experiment was executed on number of desired angles as well

9 EXAMPLE Figures

The "`figure`" environment should be used for figures. One or more images can be placed within a figure. If your figure contains third-party material, you must clearly identify it as such, as shown in the example below.

Your figures should contain a caption which describes the figure to the reader.

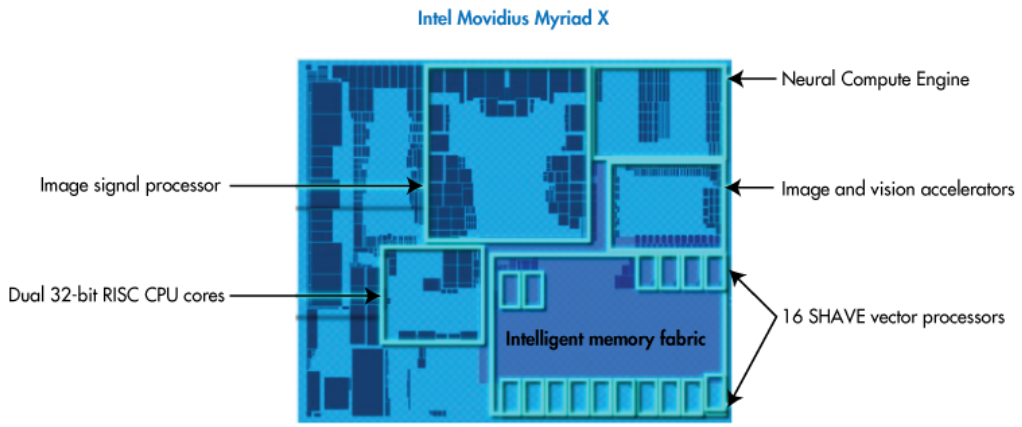

Figure 4: Diagram of the Intel Movidius Myriad X architecture. [**?**]

Figure captions are placed *below* the figure.

10 EXAMPLE Tables

This template includes the "`booktabs`" package — `https://ctan.org/pkg/booktabs` — for preparing high-quality tables.

Table captions are placed *below* the table.

Because tables cannot be split across pages, the best placement for them is typically the top of the page nearest their initial cite. To ensure this proper "floating" placement of tables, use the environment **table** to enclose the table's contents and the table caption. The contents of the table itself must go in the **tabular** environment, to be aligned properly in rows and columns, with the desired horizontal and vertical rules. Again, detailed instructions on **tabular** material are found in the *LaTeX User's Guide*.

Immediately following this sentence is the point at which Table 3 is included in the input file; compare the placement of the table here with the table in the printed output of this document.

11 EXAMPLE Math Equations

You may want to display math equations in three distinct styles: inline, numbered or non-numbered display. Each of the three are discussed in the

Non-English or Math	Frequency	Comments
Ø	1 in 1,000	For Swedish names
π	1 in 5	Common in math
\$	4 in 5	Used in business
Ψ_1^2	1 in 40,000	Unexplained usage

Table 3: Frequency of Special Characters

next sections.

11.1 Inline (In-text) Equations

A formula that appears in the running text is called an inline or in-text formula. It is produced by the **math** environment, which can be invoked with the usual `\begin ... \end` construction or with the short form `$ ... $`. You can use any of the symbols and structures, from α to ω, available in LaTeX [**?**]; this section will simply show a few examples of in-text equations in context. Notice how this equation: $\lim_{n\to\infty} x = 0$, set here in in-line math style, looks slightly different when set in display style. (See next section).

11.2 Display Equations

A numbered display equation—one set off by vertical space from the text and centered horizontally—is produced by the **equation** environment. An unnumbered display equation is produced by the **displaymath** environment.

Again, in either environment, you can use any of the symbols and structures available in LaTeX; this section will just give a couple of examples of display equations in context. First, consider the equation, shown as an inline equation above:

$$\lim_{n\to\infty} x = 0 \tag{1}$$

Notice how it is formatted somewhat differently in the **displaymath** environment. Now, we'll enter an unnumbered equation:

$$\sum_{i=0}^{\infty} x + 1$$

and follow it with another numbered equation:

$$\sum_{i=0}^{\infty} x_i = \int_0^{\pi+2} f \tag{2}$$

just to demonstrate LaTeX's able handling of numbering.

12 EXAMPLE Code Listings

You may want to highlight a piece of code. For this you can use the `lstlisting` environment.

Example of this can be found in Listing 1. In the case of languages other than Python you can use the `language` directive of the `lstlisting` environment, which will fix the formatting and code highlighting accordingly.

```
for num in range(1, 101):
    if num % 15 == 0:
        print("FizzBuzz")
    elif num % 3 == 0:
        print("Fizz")
    elif num % 5 == 0:
        print("Buzz")
    else:
        print(num)
```

Listing 1: Example Python listing (FizzBuzz)

13 Citations and Bibliographies

Authors' names should be complete — use full first names ("Donald E. Knuth") not initials ("D. E. Knuth") — and the salient identifying features of a reference should be included: title, year, volume, number, pages, article DOI, etc.

The bibliography is included in your source document with these two commands, placed just before the `\end{document}` command:

```
  \bibliographystyle{IEEEtran}
\bibliography{IEEEabrv,cites}
```

where "`cites`" is the name of the bibliography file without the suffix.

Some examples. A paginated journal article [?], an enumerated journal article [?], a reference to an entire issue [?], a monograph (whole book) [?], a monograph/whole book in a series (see 2a in spec. document) [?], a divisible-book such as an anthology or compilation [?] followed by the same example, however we only output the series if the volume number is given [?] (so Editor00a's series should NOT be present since it has no vol. no.), a chapter in a divisible book [?], a chapter in a divisible book in a series [?], a multi-volume work as book [?], a couple of articles in a proceedings (of a conference, symposium, workshop for example) (paginated proceedings article) [? ?], a proceedings article with all possible elements [?], an example of an enumerated proceedings article [?], an informally published work [?], a couple of preprints [? ?], a doctoral dissertation [?], a master's thesis: [?], an online document / world wide web resource [? ? ?], a video game (Case 1) [?] and (Case 2) [?] and [?] and (Case 3) a patent [?], work accepted for publication [?], 'YYYYb'-test for prolific author [?] and [?]. Other cites might contain 'duplicate' DOI and URLs (some SIAM articles) [?]. Boris / Barbara Beeton: multi-volume works as books [?] and [?]. A couple of citations with DOIs: [? ?]. Online citations: [? ? ?]. Artifacts: [?] and [?].

References

[1] Emax ECO II Series 2004 1600KV 2000KV 2400KV 3000KV Brushless Motor for RC Drone FPV Racing. URL https://emaxmodel.com/products/emax-eco-ii-series-2004-1600kv-2000kv-2400kv-3000kv-brushless-motor-for-rc-dr

[2] Find Your Perfect FPV Drone Size for Optimal Use. URL https://vibms.com/what-size-fpv-drone-should-i-get/.

[3] Gömböc. URL https://en.wikipedia.org/w/index.php?title=G%C3%B6mb%C3%B6c&oldid=1342148896.

[4] LiPo batteries for FPV drones explained: Voltage, sag, charging and storage basics. URL https://chinahobbyline.com/blogs/news/lipo-batteries-for-fpv-drones-explained.

[5] Holybro Pixhawk 6C. URL https://docs.px4.io/main/en/flight_controller/pixhawk6c.

[6] Pixhawk 6C 6C Mini Flight Controller — Copter documentation. URL https://ardupilot.org/copter/docs/common-holybro-pixhawk6C.html.

[7] Technical Specification | Holybro. URL https://docs.holybro.com/autopilot/pixhawk-6c/technical-specification.

[8] The Ultimate FPV Drone Motors Guide 2025, . URL https://www.ligpower.com/blog/the-ultimate-fpv-drone-motor-guide.html. info on number of cells, voltage sag, ratio, efficiency
.

[9] Ultimate Guide to Drone Cages: Types & Uses, . URL https://www.flyability.com/blog/drone-cage.

[10] Understanding LiPo Battery Internal Resistance - Hanery, . URL https://www.hanery.com/blog/lipo-battery-internal-resistance/.

[11] Understanding LiPo battery voltage for optimal performance, . URL https://www.large-battery.com/blog/lipo-battery-voltage-guide/.

[12] Install Docker Engine on Ubuntu, 10:48:12 +0200 +0200. URL https://docs.docker.com/engine/install/ubuntu/.

[13] Post-installation steps, 15:53:34 +0200 +0200. URL https://docs.docker.com/engine/install/linux-postinstall/.

[14] Innov8tive Designs. How to use multirotor motor performance data charts. URL https://innov8tivedesigns.com/images/specs/Prop-Chart-Instructions-B.pdf.

[15] Drone24Hours. T-motor f2004 1700kv. URL https://drone24hours.com/products/t-motor-f2004-1700kv.

[16] prefix=de useprefix=false family=Azambuja, given=Ricardo, Hassan Fouad, Yann Bouteiller, Charles Sol, and Giovanni Beltrame. When Being Soft Makes You Tough: A Collision-Resilient Quadcopter Inspired by Arthropods' Exoskeletons. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 7854–7860. doi: 10.1109/ICRA46639.2022.9811841. URL http://arxiv.org/abs/2103.04423. Comment: Author's version of the paper presented at ICRA 2022 (still waiting for the conference proceedings to be online). Added acknowledgements that were previously removed to fit 6+n ICRA 2022 page limits.

[17] Victor Gomez, Nicolas Gomez, Jorge Rodas Benítez, Enrique Paiva, Maarouf Saad, and Raul Gregor. Pareto Optimal PID Tuning for Px4-Based Unmanned Aerial Vehicles by Using a Multi-Objective Particle Swarm Optimization Algorithm. 7. doi: 10.3390/aerospace7060071.

[18] Alexander Niclas Lerche and Andres Faina. SpinFly: UAV System for Active Inspection in the Pharmaceutical Industry.

[19] Al Mahmud. Development of Collision Resilient Drone for Flying in Cluttered Environment. URL https://researchrepository.wvu.edu/etd/8022.

[20] MECHTEX. Understanding the Relationship Between KV Rating and Drone Performance. URL https://mechtex.com/blog/understanding-the-relationship-between-kv-rating-and-drone-performance.

[21] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240. IEEE. ISBN 978-1-4799-6923-4. doi: 10.1109/ICRA.2015.7140074. URL http://ieeexplore.ieee.org/document/7140074/.

[22] Yash Mulgaonkar, Anurag Makineni, Luis Guerrero-Bonilla, and Vijay Kumar. Robust Aerial Robot Swarms Without Collision Avoidance. 3 (1):596–603. ISSN 2377-3766. doi: 10.1109/LRA.2017.2775699. URL https://ieeexplore.ieee.org/document/8115305/.

[23] Oscar. How to Choose FPV Drone Motors - Considerations and Best Motor Recommendations, . URL https://oscarliang.com/motors/.

[24] Oscar. Using LiPo Batteries for FPV Drones: Beginner's Guide with Top Product Recommendations, . URL https://oscarliang.com/lipo-battery-guide/.

[25] Recursion Labs. 6s vs 4s - detailed head to head automated motor bench testing. URL https://www.youtube.com/watch?v=Vy_JMJNG31Y.

[26] Tyto Robotics. Drone Design - Calculations and Assumptions. URL https://www.tytorobotics.com/blogs/articles/the-drone-design-loop-for-brushless-motors-and-propellers.

[27] Sarah Simpson. Latest Pixhawk Open Standards Available | UST. URL https://www.unmannedsystemstechnology.com/2023/01/latest-pixhawk-open-standards-available/.